

3. LOGIC CONCEPTS

PROPOSITIONAL CALCULUS (PC)

Propositional Calculus (PC) refers to a language of propositions in which a set of rules are used to combine simple propositions to form compound propositions with the help of certain logical operators. These logical operators are often called connectives. Examples of some connectives are not ($\sim$), and ($\wedge$), or ($\vee$), implies ($\rightarrow$) & equivalence ($\leftrightarrow$). A well defined formula is defined as a symbol or a string of symbols generated by the formal grammar of a formal language.

Properties of a well formed formula in PC:

- The smallest unit (or an atom) is considered to be a well formed formula.
- If α is a well-formed formula, then $\sim\alpha$ is a well-formed formula.
- If α and β are well formed formulae, then $(\alpha \wedge \beta)$, $(\alpha \vee \beta)$, $(\alpha \rightarrow \beta)$, $(\alpha \leftrightarrow \beta)$ are also well-formed formulae.

Truth Table:

In PC, a truth table is used to provide operational definitions of important logical operators. The logical constants are true & false which are represented by T & F. Let us assume A, B, C, ... are propositioned symbols.

A	B	$\sim A$	$A \wedge B$	$A \vee B$	$A \rightarrow B$	$A \leftrightarrow B$
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

Example: Compute the truth value of α : $(A \vee B) \wedge (\sim B \rightarrow A)$ using truth table approach.

Solution: Using the Truth Table approach, compute truth values of $(A \vee B)$ and $(\sim B \rightarrow A)$ and then compute for the final expression $(A \vee B) \wedge (\sim B \rightarrow A)$.

A	B	$A \vee B$	$\sim B$	$\sim B \rightarrow A$	α
T	T	T	F	T	T
T	F	T	T	T	T
F	T	T	F	T	T
F	F	F	T	F	F

Equivalence Laws:

Equivalence relations (or laws) are used to reduce or simplify a given well-formed formula or to derive a new formula from the existing formula. Some of the important equivalence laws are:

Name of Relation	Equivalence Relations
Commutative Law	$A \vee B \cong B \vee A$ $A \wedge B \cong B \wedge A$
Associative Law	$A \vee (B \vee C) \cong (A \vee B) \vee C$

	$A \wedge (B \wedge C) \cong (A \wedge B) \wedge C$
Double Negation	$\sim(\sim A) \cong A$
Distributive Laws	$A \vee (B \wedge C) \cong (A \vee B) \wedge (A \vee C)$ $A \wedge (B \vee C) \cong (A \wedge B) \vee (A \wedge C)$
De Morgan's Law	$\sim(A \vee B) \cong \sim A \wedge \sim B$ $\sim(A \wedge B) \cong \sim A \vee \sim B$
Absorption Laws	$A \vee (A \wedge B) \cong A$ $A \wedge (A \vee B) \cong A$ $A \vee (\sim A \wedge B) \cong A \vee B$ $A \wedge (\sim A \vee B) \cong A \wedge B$
Idempotence	$A \vee A \cong A$ $A \wedge A \cong A$
Excluded Middle Law	$A \vee \sim A \cong T \text{ (True)}$
Contradiction Law	$A \wedge \sim A \cong F \text{ (False)}$
Commonly used equivalence relations	$A \vee F \cong A$ $A \vee T \cong T$ $A \wedge T \cong A$ $A \wedge F \cong F$ $A \rightarrow B \cong \sim A \vee B$ $A \leftrightarrow B \cong (A \rightarrow B) \wedge (B \rightarrow A)$ $\cong (A \wedge B) \vee (\sim A \wedge \sim B)$

Let us verify the absorption law $A \vee (A \wedge B) \cong A$ using the Truth Table approach

A	B	$A \wedge B$	$A \vee (A \wedge B)$
T	T	T	T
T	F	F	T
F	T	F	F
F	F	F	F

PROPOSITIONAL LOGIC (PL)

Propositional Logic (or) prop Logic deals with the validity, satisfiability (also called consistency) and unsatisfiability (inconsistency) of a formula and the derivation of a new formula using equivalence laws. Each row of a truth table for a given formula α is called its interpretation under which the value of a formula may be true or false.

- A formula α is said to be a tautology if and only if the value of α is true for all its interpretations.

Now, the validity, satisfiability & unsatisfiability of a formula may be determined on the basis of the following conditions:

- A formula α is said to be valid if and only if it is a tautology.
- A formula α is said to be satisfiable if there exists at least one interpretation for which α is true.
- A formula α is said to be unsatisfiable if the value of α is false under all interpretations.

Example: Show that the following is a valid argument: “If it is humid then it will rain and since it is humid today it will rain”

Solution: Let us symbolize each part of the English sentence by propositional atoms as follows:

A: It is humid

B: It will rain

Now, the formula (α) corresponding to the given sentence: “If it is humid then it will rain and since it is humid today it will rain” may be written as

$$\alpha: [(A \rightarrow B) \wedge A] \rightarrow B$$

The truth table for $\alpha: [(A \rightarrow B) \wedge A] \rightarrow B$ is as follows

A	B	$A \rightarrow B = X$	$X \wedge A = Y$	$Y \rightarrow B$
T	T	T	T	T
T	F	F	F	T
F	T	T	F	T
F	F	T	F	T

Truth table approach is a simple & straight forward method & is extremely useful at presenting an overview of all the truth values in a given situation. Although it is an easy method for evaluating consistency, in consistency (or) validity of a formula, the limitation of this method lies in the fact that the size of the truth table grows exponentially. If a formula contains n atoms, then the truth table will contain 2^n entries.

For example, if we have to show that a formula $\alpha: (A \wedge B \wedge C \wedge D) \rightarrow (B \vee E)$ is valid using the truth table approach, then we need to construct the table containing 32 rows & complete the truth values.

There are other methods which can help in proving the validity of the formula directly. Some other methods that are concerned with proofs & deductions are

- Natural Deductive System
- Axiomatic System
- Semantic Tableaux Method
- Resolution Refutation Method

NATURAL DEDUCTION SYSTEM (NDS)

Natural Deduction System (NDS) is thus called because of the fact that it mimics the pattern of natural reasoning. The system is based on a set of deductive inference rules. It has about 10 deductive inference rules. They are as follows

Rule Name	Symbol	Rule	Description
Introducing $\wedge$	(I: $\wedge$)	If $A_1, \dots, A_n$ then $A_1 \wedge \dots \wedge A_n$	If $A_1, \dots, A_n$ are true, then their conjunction $A_1 \wedge \dots \wedge A_n$ is also true.
Eliminating $\wedge$	(E: $\wedge$)	If $A_1 \wedge \dots \wedge A_n$ then A_i ($1 \leq i \leq n$)	If $A_1 \wedge \dots \wedge A_n$ is true, then any A_i is also true.
Introducing $\vee$	(I: $\vee$)	If any A_i ($1 \leq i \leq n$) then $A_1 \vee \dots \vee A_n$	If any A_i ($1 \leq i \leq n$) is true, then $A_1 \vee \dots \vee A_n$ is also true.
Eliminating $\vee$	(E: $\vee$)	If $A_1 \vee \dots \vee A_n, A_1 \rightarrow A, \dots, A_n \rightarrow A$ then A	If $A_1 \vee \dots \vee A_n, A_1 \rightarrow A, A_2 \rightarrow A, \dots$, and $A_n \rightarrow A$ are true, then A is true.
Introducing $\rightarrow$	(I: $\rightarrow$)	If from $\alpha_1, \dots, \alpha_n$ infer β is proved then $\alpha_1 \wedge \dots \wedge \alpha_n \rightarrow \beta$ is proved	If given that $\alpha_1, \alpha_2, \dots$, and α_n are true and from these we deduce β then $\alpha_1 \wedge \dots \wedge \alpha_n \rightarrow \beta$ is also true.

Rule Name	Symbol	Rule	Description
Eliminating $\rightarrow$	(E: $\rightarrow$)	If $A_1 \rightarrow A, A_1$, then A	If $A_1 \rightarrow A$ and A_1 are true then A is also true. This is called <i>Modus Ponens</i> rule.
Introducing $\leftrightarrow$	(I: $\leftrightarrow$)	If $A_1 \rightarrow A_2, A_2 \rightarrow A_1$ then $A_1 \leftrightarrow A_2$	If $A_1 \rightarrow A_2$ and $A_2 \rightarrow A_1$ are true then $A_1 \leftrightarrow A_2$ is also true.
Elimination $\leftrightarrow$	(E: $\leftrightarrow$)	If $A_1 \leftrightarrow A_2$ then $A_1 \rightarrow A_2, A_2 \rightarrow A_1$	If $A_1 \leftrightarrow A_2$ is true then $A_1 \rightarrow A_2$ and $A_2 \rightarrow A_1$ are true
Introducing $\sim$	(I: $\sim$)	If from A infer $A_1 \wedge \sim A_1$ is proved then $\sim A$ is proved	If from A (which is true), a contradiction is proved then truth of $\sim A$ is also proved
Eliminating $\sim$	(E: $\sim$)	If from $\sim A$ infer $A_1 \wedge \sim A_1$ is proved then A is proved	If from $\sim A$, a contradiction is proved then truth of A is also proved

Example 1: Prove that $A \wedge (B \vee C)$ is deduced from $A \wedge B$.

Solution: The theorem in NDS can be written as from $A \wedge B$ infer $A \wedge (B \vee C)$ in NDS. We can prove the theorem as follows

Description	Formula	Comments
<i>Theorem</i>	<i>from $A \wedge B$ infer $A \wedge (B \vee C)$</i>	<i>To be proved</i>
Hypothesis (given)	$A \wedge B$	1
E: $\wedge$ (1)	A	2
E: $\wedge$ (1)	B	3
I: $\vee$ (3)	$B \vee C$	4
I: $\wedge$ (2, 4)	$A \wedge (B \vee C)$	Proved

Example 2: Prove the theorem infer $[(A \rightarrow B) \wedge (B \rightarrow C)] \rightarrow (A \rightarrow C)$

Solution: The theorem infer $[(A \rightarrow B) \wedge (B \rightarrow C)] \rightarrow (A \rightarrow C)$ is reduced to the theorem from $(A \rightarrow B), (B \rightarrow C)$ infer $(A \rightarrow C)$ using deduction theorem. Further to prove ' $A \rightarrow C$ ' we will have to prove a sub theorem from A infer C . The proof of the theorem is as follows

Description	Formula	Comments
<i>Theorem</i>	<i>from $A \rightarrow B, B \rightarrow C$ infer $A \rightarrow C$</i>	To be proved
Hypothesis 1	$A \rightarrow B$	1
Hypothesis 2	$B \rightarrow C$	2
Sub-theorem	<i>from A infer C</i>	3
Hypothesis	A	3.1
E: $\rightarrow (1, 3.1)$	B	3.2
E: $\rightarrow (2, 3.2)$	C	3.3
I: $\rightarrow (3)$	$A \rightarrow C$	Proved

AXIOMATIC SYSTEM

The Axiomatic System is based on a set of 3 axioms & 1 rule called Modus Ponens (MP). Although it is minimal in structure, it is as powerful as truth table & NDS. In this system, only 2 logical operators not ($\sim$) & implies ($\rightarrow$) are allowed to form a formula. It should be noted that other logical operators such as and ($\wedge$), or ($\vee$) & equivalence ($\leftrightarrow$) can be easily expressed in terms of not ($\sim$) & implies ($\rightarrow$) using equivalence laws. For example,

- $A \wedge B \cong \sim (\sim A \vee \sim B) \cong \sim (A \rightarrow \sim B)$
- $A \vee B \cong \sim A \rightarrow B$
- $A \leftrightarrow B \cong (A \rightarrow B) \wedge (B \rightarrow A) \cong \sim [(A \rightarrow B) \rightarrow \sim (B \rightarrow A)]$

In axiomatic system, α , β & γ are well-formed formulae.

Axiom 1: $\alpha \rightarrow (\beta \rightarrow \alpha)$

Axiom 2: $(\alpha \rightarrow (\beta \rightarrow \gamma)) \rightarrow ((\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \gamma))$

Axiom 3: $(\sim \alpha \rightarrow \sim \beta) \rightarrow (\beta \rightarrow \alpha)$

Modus Ponens (MP) Rule: Hypotheses: $\alpha \rightarrow \beta$ and α Consequent: β

Interpretation of Modus Ponens Rule: Given that $\alpha \rightarrow \beta$ and α are hypotheses (assumed to be true), β is inferred (true) as a consequent.

Example 1: Establish that $A \rightarrow C$ is a deductive consequence of $\{A \rightarrow B, B \rightarrow C\}$, i.e., $\{A \rightarrow B, B \rightarrow C\} \vdash (A \rightarrow C)$

Description	Formula	Comments
<i>Theorem</i>	$\{A \rightarrow B, B \rightarrow C\} \vdash (A \rightarrow C)$	<i>Prove</i>
Hypothesis 1	$A \rightarrow B$	1
Hypothesis 2	$B \rightarrow C$	2
Instance of Axiom 1	$(B \rightarrow C) \rightarrow [A \rightarrow (B \rightarrow C)]$	3
MP (2, 3)	$[A \rightarrow (B \rightarrow C)]$	4
Instance of Axiom 2	$[A \rightarrow (B \rightarrow C)] \rightarrow [(A \rightarrow B) \rightarrow (A \rightarrow C)]$	5
MP (4, 5)	$(A \rightarrow B) \rightarrow (A \rightarrow C)$	6
MP (1, 6)	$(A \rightarrow C)$	<i>Proved</i>

Example 2: Prove $\vdash \sim A \rightarrow (A \rightarrow B)$ by using deduction theorem.

Solution: If we can prove $\{\sim A\} \vdash (A \rightarrow B)$ then using deduction theorem, we have proved $\vdash \sim A \rightarrow (A \rightarrow B)$. The proof is shown below

Description	Formula	Comments
<i>Theorem</i>	$\{\sim A\} \vdash (A \rightarrow B)$	<i>Prove</i>
Hypothesis 1	$\sim A$	1
Instance of Axiom 1	$\sim A \rightarrow (\sim B \rightarrow \sim A)$	2
MP (1, 2)	$(\sim B \rightarrow \sim A)$	3
Instance of Axiom 3	$(\sim B \rightarrow \sim A) \rightarrow (A \rightarrow B)$	4
MP (3, 4)	$(A \rightarrow B)$	<i>Proved</i>

Note: If Σ is an empty set & α is deduced, then we can write $\vdash \alpha$. In this case, α is deduced from axioms only & no hypotheses are used.

SEMANTIC TABLEAU SYSTEM IN PROPOSITIONAL LOGIC

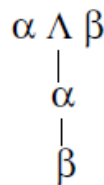
In both Natural Deduction & Axiomatic Systems, forward chaining approach is used for constructing proofs & derivations. In this approach we start proofs (or) derivations from a given set of hypotheses (or) axioms. In Semantic Tableau System, proofs follow backward chaining approach. In this method, a set of rules are applied systematically on a formula or a set of formulae in order to establish consistency or inconsistency.

Semantic Tableau Rules:

The Semantic Tableau Rules are given below where α & β are two formulae

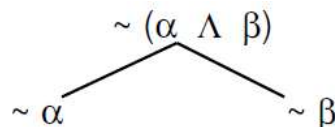
Rule 1: A tableau for a formula $(\alpha \wedge \beta)$ is constructed by adding both α and β to the same path (branch).

$\alpha \wedge \beta$ is true if both α and β are true

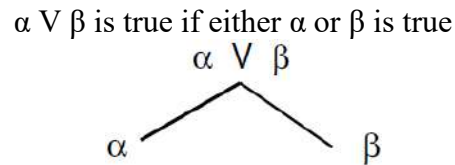


Rule 2: A tableau for a formula $\sim (\alpha \wedge \beta)$ is constructed by adding two new paths, one containing $\sim \alpha$ and other containing $\sim \beta$.

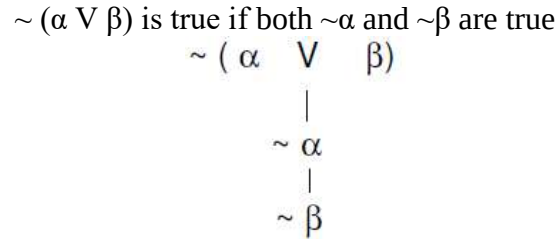
$\sim (\alpha \wedge \beta)$ is true if either $\sim \alpha$ or $\sim \beta$ is true



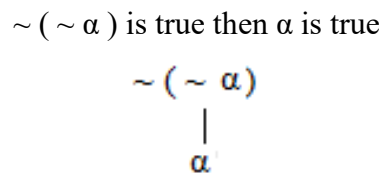
Rule 3: A tableau for a formula $(\alpha \vee \beta)$ is constructed by adding two new paths, one containing α and other containing β .



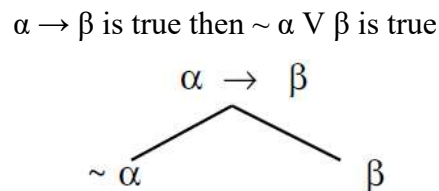
Rule 4: A tableau for a formula $\sim(\alpha \vee \beta)$ is constructed by adding both $\sim\alpha$ and $\sim\beta$ to the same path.



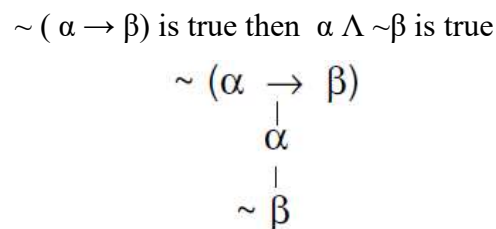
Rule 5: A tableau for $\sim(\sim\alpha)$ is constructed by adding α on the same path.



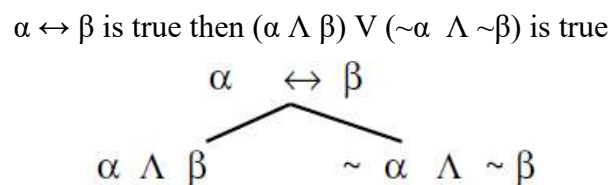
Rule 6: A tableau for a formula $\alpha \rightarrow \beta$ is constructed by adding two new paths: one containing $\sim\alpha$ and the other containing β .



Rule 7: A tableau for a formula $\sim(\alpha \rightarrow \beta)$ is constructed by adding both α & $\sim\beta$ to the same path.

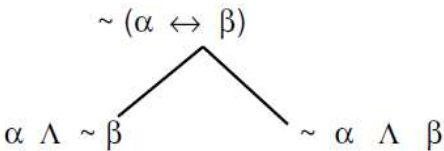


Rule 8: A tableau for a formula $\alpha \leftrightarrow \beta$ is constructed by adding two new paths: one containing $\alpha \wedge \beta$ and other $\sim\alpha \wedge \sim\beta$ which are further expanded.



Rule 9: A tableau for a formula $\sim (\alpha \leftrightarrow \beta)$ is constructed by adding two new paths: one containing $\alpha \wedge \sim \beta$ and the other $\sim \alpha \wedge \beta$ which are further expanded.

$\sim (\alpha \leftrightarrow \beta)$ is true then $(\alpha \wedge \sim \beta) \vee (\sim \alpha \wedge \beta)$ is true



Example 1: Construct a Semantic Tableau for a formula $(A \wedge \sim B) \wedge (\sim B \rightarrow C)$

Description	Formula	Line number
Tableau root	$(A \wedge \sim B) \wedge (\sim B \rightarrow C)$	1
Rule 1 (1)	$A \wedge \sim B$	2
	$\sim B \rightarrow C$	3
Rule 1 (2)	A	4
	$\sim B$	5
Rule 6 (3)	$\sim(\sim B)$ B $\vee(\text{open})$	6
Rule 3 (6)	$\times$ (closed) $\{B, \sim B\}$	

PREDICATE LOGIC

Logic: Logic is concerned with the truth of statements about the world. Generally each statement is either True or False. Logic includes

- Syntax
- Semantics
- Inference Procedure

Syntax: Specifies the symbols in the language about how they can be combined to form sentences. The facts about the world are represented as sentences in logic.

Semantic: Specifies how to assign a truth value to a sentence based on its meaning in the world. It specifies what facts a sentence refers to. A fact is a claim about the world, and it may be True or False.

Inference Procedure: Specifies methods for computing new sentences from an existing ones.

Predicate Logic: It is the study of individuals and their properties.

Conside the following set of sentences:

1. Marcus was a man.

2. Marcus was a Pompeian.
3. All Pompeians were Romans
4. Caesar was a ruler
5. All Romans were either loyal to Caesar or hated him.
6. Everyone is loyal to someone.
7. People only try to assassinate rulers they are not loyal to
8. Marcus tried to assassinate Caesar.

The facts described by these sentences can be represented as a set of WFF's (Well Formed Formula) in Predicate Logic as follows:

1. Marcus was a man.
man (Marcus)
2. Marcus was a Pompeian.
Pompeian (Marcus)
3. All Pompeians were Romans.
 $\forall x: \text{Pompeian}(x) \rightarrow \text{Roman}(x)$
4. Caesar was a ruler.
ruler (Caesar)
5. All Romans were either loyal to Caesar or hated him.
 $\forall x: \text{Roman}(x) \rightarrow \text{loyal to}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar})$
6. Everyone is loyal to someone.
 $\forall x: \exists y: \text{loyal to}(x, y)$
7. People only try to assassinate rulers they are not loyal to
 $\forall x: \forall y: \text{person}(x) \wedge \text{ruler}(y) \wedge \text{try assassinate}(x, y) \rightarrow \neg \text{loyal to}(x, y)$
8. Marcus tried to assassinate Caesar.
tryassassinate(Marcus, Caesar)

Now suppose we want to use these statements to answer the question

Was Marcus loyal to Caesar?

It seems that using 7 and 8, we should be able to prove that Marcus was not loyal to Caesar. Now let's try to produce a formal proof, reasoning backward from the desired goal:

$\neg \text{loyal to}(\text{Marcus}, \text{Caesar})$

The attempt fails, however, since there is no way to satisfy the goal `person(Marcus)` with the statements available. We know that Marcus was a man; we do not have any way to conclude from that Marcus was a person. We need to add the representation of another fact to our system, namely:

9. All men are people.

$\forall x : \text{man}(x) \rightarrow \text{person}(x)$

$$\begin{array}{c}
 \neg \text{loyalto}(\text{Marcus}, \text{Caesar}) \\
 \uparrow \quad (7, \text{substitution}) \\
 \text{person}(\text{Marcus}) \wedge \\
 \text{ruler}(\text{Caesar}) \wedge \\
 \text{tryassassinate}(\text{Marcus}, \text{Caesar}) \\
 \uparrow \quad (4) \\
 \text{person}(\text{Marcus}) \wedge \\
 \text{tryassassinate}(\text{Marcus}, \text{Caesar}) \\
 \uparrow \quad (8) \\
 \text{person}(\text{Marcus})
 \end{array}$$

Fig: An Attempt to Prove $\neg \text{loyalto}(\text{Marcus}, \text{Caesar})$

Now we can satisfy the last goal and produce a proof that Marcus was not loyal to Caesar. From this example, we see that three important issues must be addressed in the process of converting English sentences into logical statements and then using those statements to deduce new ones.

- Many English sentences are ambiguous. (Ex: 5, 6, and 7 above). Choosing the correct interpretation may be difficult.
- There is often a choice of how to represent the knowledge (1 and 7). Simple representations are desirable, but they may preclude certain kinds of reasoning.
- Even in very simple situations, a set of sentences is unlikely to contain all the information necessary to reason about the topic at hand.

Representing Instance And Isa Relationships:

The below figure shows the five sentences of the last section represented in logic in three different ways. The first part of the figure contains the representations we have already discussed. In these representations, class membership is represented with unary predicates (such as `Roman`), each of which corresponds to a class. Asserting that `p(x)` is true is equivalent to asserting that `x` is an instance (or element) of `P`. The second part of figure contains representations that use the instance predicate explicitly.

The predicate instance is a binary one, whose first argument is an object and whose second argument is a class to which the object belongs. But these representations do not use an explicit isa predicate. Instead, subclass relationships, such as that between Pompeians and Romans, are described as shown in sentence 3. The implication rule there states that if an object is an instance of subclass Pompeian then it is an instance of

superclass Roman. This rule is equivalent to subclass-superclass relationship. The third part contains representations that use both instance and isa predicates explicitly. The additional axiom describes how an instance relation and an isa relation can be combined to derive a new instance relation.

Statements as well-formed formulas (WFF's):

1. man (Marcus)
2. Pompeian (Marcus)
3. $\forall x: \text{Pompeian}(x) \rightarrow \text{Roman}(x)$
4. ruler (Caesar)
5. $\forall x: \text{Roman}(x) \rightarrow \text{loyal to}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar})$

Statement representation using INSTANCE predicate:

1. instance (Marcus, man)
2. instance (Marcus, Pompeian)
3. $\forall x: \text{instance}(x, \text{Pompeian}) \rightarrow \text{instance}(x, \text{Roman})$
4. instance (Caesar, ruler)
5. $\forall x: \text{instance}(x, \text{Roman}) \rightarrow \text{loyalto}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar})$

Statements using ISA predicate:

1. instance (Marcus, man)
2. instance (Marcus, Pompeian)
3. isa (Pompeian, Roman)
4. instance (Caesar, ruler)
5. $\forall x: \text{instance}(x, \text{Roman}) \rightarrow \text{loyalto}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar})$
6. $\forall x: \forall y: \forall z: \text{instance}(x, y) \wedge \text{isa}(y, z) \rightarrow \text{instance}(x, z)$

Computable Functions And Predicates:

In some cases we need to exempt some cases. For example that Paulus was a roman and he never hates Caesar. So in representing the statement that all roman was hated Caesar, only Paulus need to be exempted. And hence the statement will be as follows:

$$\forall x: \text{Roman}(x) \wedge \neg \text{equalto}(x, \text{Paulus}) \rightarrow \text{loyalto}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar})$$

These types of exceptions are very easy to represent in case of smaller number of statements. Suppose we want to express simple facts, such as the following greater-than and less-than relationships:

$$\begin{array}{ll} \text{gt}(1, 0) & \text{lt}(0, 1) \\ \text{gt}(2, 1) & \text{lt}(1, 2) \\ \text{gt}(3, 2) & \text{lt}(2, 3) \quad \text{and so on.} \end{array}$$

If there is very large number of predicates like the above statements we can easily represent them and more over we can also simplify them if we have additional knowledge about them. That is we can infer new statements in smaller number by inferring new statements from the old.

1. Marcus was a man.

Man (Marcus)

2. Marcus was a Pompeian.

Pompeian (Marcus)

3. Marcus was born in 40 A.D.

born (Marcus, 40)

4. All means are mortal.

$\forall x: \text{man}(x) \rightarrow \text{mortal}(x)$

5. All Pompeian's died when the volcano erupted in 79 A.D.

$\text{Erupted}(\text{volcano}, 79) \wedge \forall x: [\text{Pompeian}(x) \rightarrow \text{died}(x, 79)]$

6. No mortal lives longer than 150yers.

$\forall x: \forall t1: \forall t2: \text{mortal}(x) \wedge \text{born}(x, t1) \wedge \text{gt}(t2-t1, 150) \rightarrow \text{dead}(x, t2)$

7. It is now 1991 A.D.

now= 1991

Now we want to answer the question is “Is Marcus alive?” For example, our statements talk about dying, but they say nothing that relates to being alive, which is what the question is asking. So we add the following facts:

8. Alive means not dead.

$\forall x: \forall t: [\text{alive}(x, t) \rightarrow \neg \text{dead}(x, t)] \wedge [\neg \text{dead}(x, t) \rightarrow \text{alive}(x, t)]$

This is an example of the fact that rarely do two expressions have truly identical meanings in all circumstances.

9. If someone dies, then he is dead at all later times.

$\forall x: \forall t1: \forall t2: \text{died}(x, t1) \wedge \text{gt}(t2, t1) \rightarrow \text{dead}(x, t2)$

This representation says that one is dead in all years after the one in which one died. Now let's attempt to answer the question “Is Marcus alive?” by proving:

$\neg \text{alive}(\text{Marcus}, \text{now})$

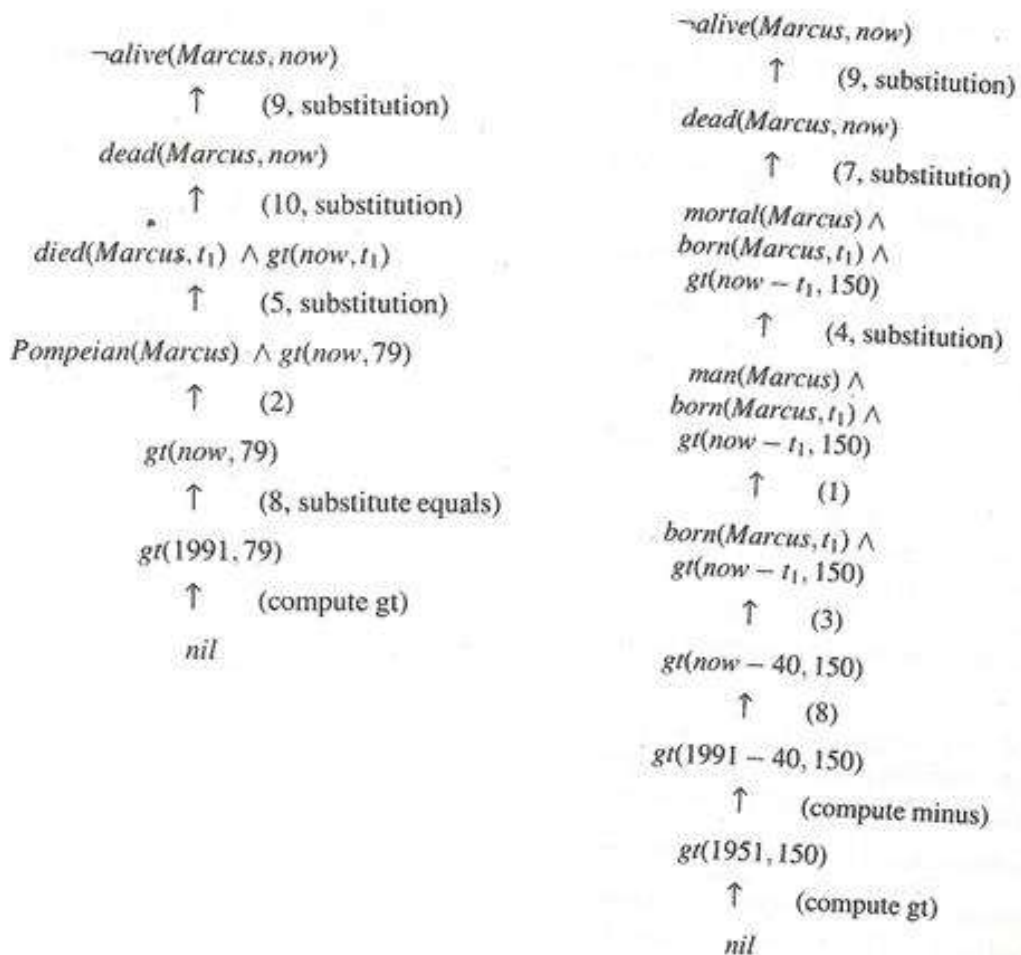


Fig: Two ways of Proving that Marcus Is Dead

RESOLUTION

Resolution is a procedure used in proving that arguments which are expressible in predicate logic are correct. Resolution is so far only defined for Propositional Logic. The resolution process can also be applied to predicate logic also. Resolution produces proofs by refutation. In other words, to prove the validity of a statement, resolution attempts to show that the negation of the statement produces a contradiction with the known statements (i.e., the statement is unsatisfiable).

The Basis of Resolution:

The resolution procedure is an iterative process; at each step, two clauses called the parent clauses containing the same literal, are compared (resolved), yielding a new clause called the resolvent, that has been inferred from them. The literal must occur in positive form in one clause and in negative form in the other. The resolvent is obtained by combining all of the literals of the parent clauses except the ones that cancel. If the clause that is produced is an empty clause, then a contradiction has been found.

Given: winter $\vee$ summer

$\neg$ winter $\vee$ cold

We can conclude: summer $\vee$ cold

In predicate logic, the situation is more complicated since we must consider all possible ways of substituting values for the variables. The theoretical basis of the resolution procedure in predicate logic is Herbrand's Theorem, which tells us the following.

- To show that a set of clauses S is unsatisfiable, it is necessary to consider only interpretations over a particular set, called the Herbrand universe of S .
- A set of clauses S is unsatisfiable if and only if finite subset of ground instances of S is unsatisfiable.

The second part of the theorem is important if there is to exist any computational procedure for proving unsatisfiability, since in a finite amount of time no procedure will be able to examine an infinite set. The first part suggests that one way to go about finding a contradiction is to try systematically the possible substitutions and see if each produces a contradiction.

So we observed here that to prove the statements using predicate logic the statements and well-formed formulas must be converted into clause form and the process will be as follows:

Conversion to Clause Form:

One method we shall examine is called resolution of requires that all statements to be converted into a normalized clause form .we define a clause as the disjunction of a number of literals. A ground clause is one in which no variables occur in the expression. A horn clause is a clause with at most one positive literal.

To transform a sentence into clause form requires the following steps:

1. Eliminate $\rightarrow$.
2. Reduce the scope of each $\neg$ to a single term.
3. Standardize variables so that each quantifier binds a unique variable.
4. Move all quantifiers to the left without changing their relative order.
5. Eliminate $\exists$ (Skolemization).
6. Drop $\forall$.
7. Convert the formula into a conjunction of disjuncts.
8. Create a separate clause corresponding to each conjunct.
9. Standardize apart the variables in the set of obtained clauses.

We describe the process of eliminating the existential quantifiers through a substitution process. This process requires that all such variables are replaced by something called Skolem functions. Sometimes ones with no arguments are called Skolem constants.

Example in Predicate logic: Suppose we know that “**all Romans who know Marcus either hate Caesar or think that any one who hates any one is crazy**”. We could represent that in the following wff:

$$\forall x: [\text{Roman}(x) \wedge \text{know}(x, \text{marcus})] \rightarrow [\text{hate}(x, \text{Caesar}) \vee (\forall y: \exists z: \text{hate}(y, z) \rightarrow \text{thinkcrazy}(x, y))]$$

1. Eliminate implies symbol using the fact that $a \rightarrow b = \neg a \vee b$.

$$\forall x: \neg [\text{Roman}(x) \wedge \text{know}(x, \text{marcus})] \vee [\text{hate}(x, \text{Caesar}) \vee (\forall y: \neg (\exists z: \text{hate}(y, z) \vee \text{thinkcrazy}(x, y)))]$$

2. Reduce the scope of each $\neg$ to a single term.

$$\forall x: [\neg \text{Roman}(x) \vee \neg \text{know}(x, \text{marcus})] \vee [\text{hate}(x, \text{Caesar}) \vee (\forall y: \forall z: \neg \text{hate}(y, z) \vee \text{thinkcrazy}(x, y))]$$

$$[\neg (\neg p) = p]. [\neg (a \wedge b) = \neg a \vee \neg b]. [\neg \exists x: p(x) = \forall x: \neg p(x)]. [\neg \forall x: p(x) = \exists x: \neg p(x)].$$

3. Standardize variables so that each quantifier binds a unique variable. Since variables are just dummy names, this process cannot affect the truth value of the wff.

$$\forall x: P(x) \vee \forall x: Q(x) \text{ would be converted into } \forall x: P(x) \vee \forall y: Q(y)$$

4. Move all quantifiers to the left of the formula with out changing their relative order.

$$\forall x: \forall y: \forall z: [\neg \text{Roman}(x) \vee \neg \text{know}(x, \text{marcus})] \vee [\text{hate}(x, \text{Caesar}) \vee \neg \text{hate}(y, z) \vee \text{thinkcrazy}(x, y)]$$

5. Eliminate existential quantifiers.

In our example, there are no existential quantifiers at step 4. There is no need to eliminate those quantifiers. If those quantifiers occur then use Skolem functions to eliminate those quantifiers. For ex:

$$\exists y: \text{President}(y) \text{ can be transformed into the formula } \text{President}(S1)$$

$$\forall x: \exists y: \text{father-of}(y, x) \text{ would be transformed into } \forall x: \text{father-of}(S2(x), x)$$

6. Drop the prefix. From (4).

$$[\neg \text{Roman}(x) \vee \neg \text{know}(x, \text{Marcus})] \vee [\text{hate}(x, \text{Caesar}) \vee (\neg \text{hate}(y, z) \vee \text{thinkcrazy}(x, y))]$$

7. Convert the matrix into a conjunction of disjuncts. In our problem that are no (ands) disjuncts. So use associative property. $(a \vee b) \vee c = a \vee (b \vee c)$.

$$\neg \text{Roman}(x) \vee \neg \text{know}(x, \text{Marcus}) \vee \text{hate}(x, \text{Caesar}) \vee \neg \text{hate}(y, z) \vee \text{thinkcrazy}(x, y).$$

$$\text{Ex: } [\text{winter} \vee (\text{summer} \wedge \text{wearingsandals})] \vee [\text{wearing boots} \vee (\text{summer} \wedge \text{wearingsandals})]$$

$$(\text{winter} \vee \text{summer}) \wedge$$

$$(\text{winter} \vee \text{wearingsandals}) \wedge$$

$$(\text{wearing boots} \vee \text{summer}) \wedge$$

$$(\text{wearing boots} \vee \text{wearingsandals})$$

8. Create a separate clause corresponding to each conjunct.

$$(\text{winter} \vee \text{summer}), (\text{winter} \vee \text{wearingsandals}), (\text{wearing boots} \vee \text{summer}), (\text{wearing boots} \vee \text{wearingsandals})$$

Resolution in Propositional Logic:

We first present the resolution procedure for propositional logic. We then expand it to include predicate logic. In propositional logic, the procedure for producing a proof by resolution of proposition P with respect to a set of axioms F is the following.

Algorithm: Resolution Propositional

1. Convert all the propositions of F to clause form.
2. Negate P and convert the result to clause form. Add it to the set of clauses obtained in 1.
3. Repeat until either a contradiction is found or no progress can be made:
 - a) Select two clauses. Call these the parent clauses.
 - b) Resolve them together. The resulting clause, called the resolvent, will be the disjunction of all of the literals of both of the parent clauses with the following exception: if there are any pairs of literals L and $\neg L$ such that one of the parent clauses contains L and other contains $\neg L$, then select one such pair and eliminate both L and $\neg L$ from the resolvent.
 - c) If the resolvent is the empty clause, then a contradiction has been found. If it is not, then add it to the set of clauses available to the procedure.

A Few Facts in Propositional Logic:

<u>Given Axioms</u>	<u>Clause Form</u>	
P	P	(1)
$(P \wedge Q) \rightarrow R$	$\neg P \vee \neg Q \vee R$	(2)
$(S \vee T) \rightarrow Q$	$\neg S \vee \neg T \vee Q$	(3)
	$\neg T \vee Q$	(4)
T	T	(5)

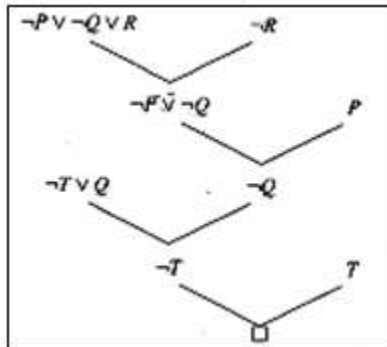


Fig: Resolution in Propositional Logic

The Unification Algorithm:

In propositional logic, easily determine two literals and its contradictions. Simply look for L and $\neg L$. In predicate logic, this matching process is more complicated since the arguments of the predicates must be considered. For ex: $\text{man}(\text{John})$ and $\neg \text{man}(\text{John})$ is a contradiction. While $\text{man}(\text{John})$ and $\neg \text{man}(\text{John})$ is not. Thus in order to determine contradictions, we need a matching procedure that compares two literals and discovers whether there exists a set of substitutions that makes them identical. The recursive procedure that will do the above called the **unification algorithm**.

The basic idea of unification is very simple. To attempt to unify two literals, we first check,

1. If their initial predicate symbols are the same. If so, we can proceed, else we can't unify them. Ex. try $\text{assassinate}(\text{Marcus}, \text{Caesar})$. $\text{hate}(\text{Marcus}, \text{Caesar})$.
2. If the predicate symbols match, we must check the arguments, one pair at a time. If the first matches, we can continue with the second, and so on. To test each argument pair we can simply call the unification procedure recursively. The matching rules are
 - Different constants or predicates cannot match; identical ones can.
 - A variable can match another variable, any constant, or a predicate expression.

With the restriction that the predicate expression must not contain any instances of the variables being matched. For example, suppose we want to unify the expressions

$$P(x, x) \text{ and } P(y, z)$$

The two instances of P match fine. Next we compare x and y , and decides that if we substitute y for x , they could match. We will write that substitute y for $x \rightarrow y/x$.

The next match x and z , we produce the substitution z for $x \rightarrow z/x$. But we cannot substitute both y and z for x , we have not produced a consistent substitution. To avoid this we'll make the substitution y/x through out the literals.

$$P(y, y) \text{ and } P(y, z) \text{ (substitution of } y/x)$$

Now, we can attempt to unify arguments y and z which succeeds the substitution z/y

Algorithm: Unify ($L1, L2$)

1. If $L1$ and $L2$ are both variables or constants, then:
 - a) If $L1$ and $L2$ are identical, then return NIL.
 - b) Else if $L1$ is a variable, then if $L1$ occurs in $L2$ then return {FAIL}, else return($L2/L1$).
 - c) Else if $L2$ is a variable, then if $L2$ occurs in $L1$ then return {FAIL}, else return($L1/L2$).
 - d) Else return {FAIL}.
2. If the initial predicate symbols in $L1$ and $L2$ are not identical, then return {FAIL}.

3. If L1 and L2 have a different number of arguments, then return {FAIL}.
4. Set SUBST to NIL.
5. For $i \leftarrow 1$ to number of arguments
 - a) Call Unify with the i th argument of L1 and i th argument of L2, putting result in S.
 - b) If S contains FAIL then return {FAIL}.
 - c) If S is not equal to NIL then:
 - i. Apply S to the remainder of both L1 and L2.
 - ii. SUBST:= APPEND (S, SUBST).
6. Return SUBST.

Resolution in Predicate Logic:

Algorithm: Resolution

1. Convert all the statements of F to clause form.
2. Negate P and convert the result to clause form. Add it to the set of clauses obtained in 1.
3. Repeat until either a contradiction is found or no progress can be made, or a predetermined amount of effort has been expended.
 - a) Select two clauses. Call these the parent clauses.
 - b) Resolve them together. The resolve will be the disjunction of all of the literals of both parent clauses with appropriate substitutions performed and with the following exception: if there is one pair of literals T1 and $\neg T2$ such that one of the parent clauses contains T1 and other contains T2 and if T1 and T2 are unifiable, then neither T1 nor T2 complementary literals. Use the substitution produced by the unification to create the resolvent. If there is more than one pair of complementary literals, only one pair should be omitted from the resolvent.
 - c) If the resolvent is the empty clause, then a contradiction has been found. If it is not, then add it to the set of clauses available to the procedure.

⚡ Axioms in clause form:

1. *man*(Marcus)
2. *Pompeian*(Marcus)
3. $\neg \textit{Pompeian}(x_1) \vee \textit{Roman}(x_1)$
4. *Ruler*(Caesar)
5. $\neg \textit{Roman}(x_2) \vee \textit{loyalto}(x_2, \textit{Caesar}) \vee \textit{hate}(x_2, \textit{Caesar})$
6. *loyalto*(x_3 , $f1(x_3)$)
7. $\neg \textit{man}(x_4) \vee \neg \textit{ruler}(y_1) \vee \neg \textit{tryassassinate}(x_4, y_1) \vee \textit{loyalto}(x_4, y_1)$
8. *tryassassinate*(Marcus, Caesar)

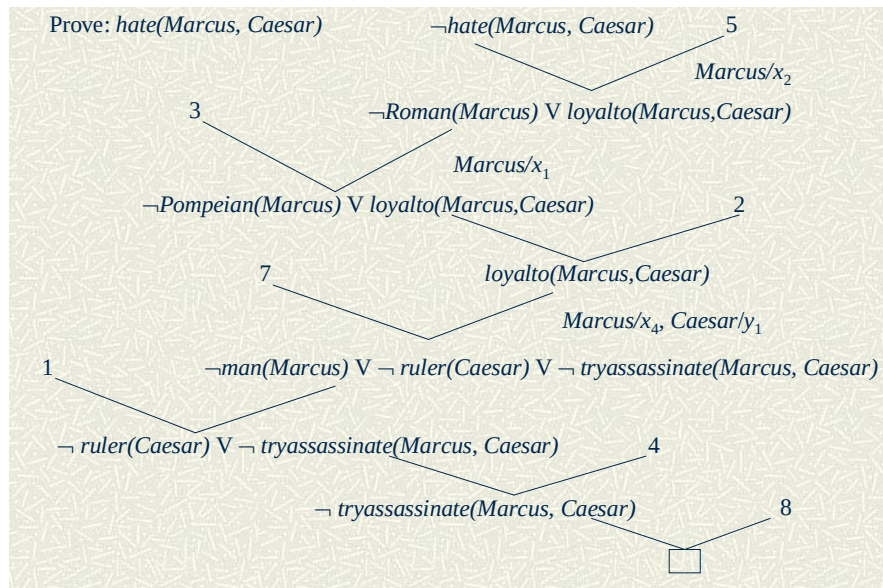


Fig: A Resolution Proof

9. persecute (x, y) $\rightarrow$ hate (y, x)

10. hate (x, y) $\rightarrow$ persecute (y, x)

Converting to clause form we get

9. $\neg$ persecute (x_5, y_2) $\vee$ hate (y_2, x_5)

10. $\neg$ hate (x_6, y_3) $\vee$ persecute (y_3, x_6)

Now to detect that there is no contradiction we must discover that the only resolvents that can be generated before. In other words, although we can generate resolvents, we can generate no new ones.

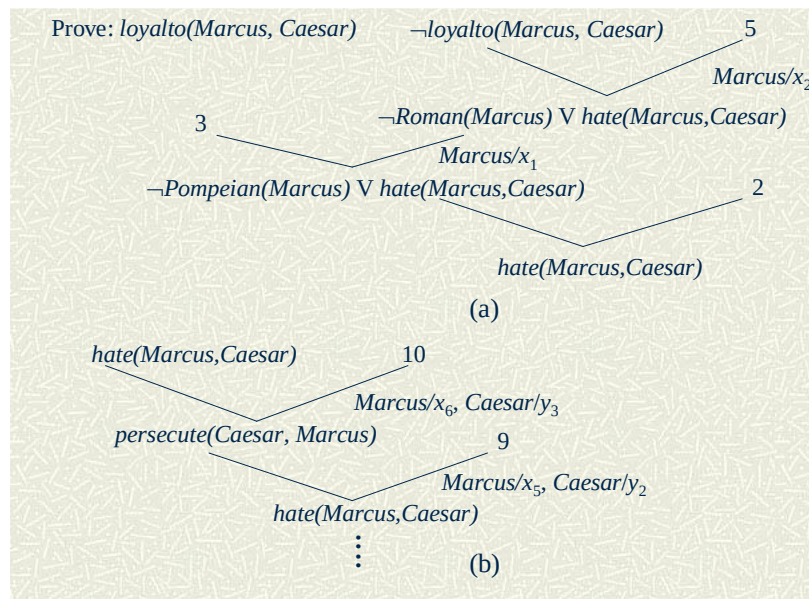


Fig: An Unsuccessful Attempt at Resolution

Axioms in clause form:

1. $man(Marcus)$
2. $Pompeian(Marcus)$
3. $born(Marcus, 40)$
4. $\neg man(x_1) \vee mortal(x_1)$
5. $\neg Pompeian(x_2) \vee died(x_2, 79)$
6. $erupted(volcano, 79)$
7. $\neg mortal(x_3) \vee \neg born(x_3, t_1) \vee \neg gt(t_2 - t_1, 150) \vee dead(x_3, t_2)$
8. $now = 1991$
- 9a. $\neg alive(x_4, t_3) \vee \neg dead(x_4, t_3)$
- 9b. $dead(x_5, t_4) \vee alive(x_5, t_4)$
10. $\neg died(x_6, t_5) \vee \neg gt(t_6, t_5) \vee dead(x_6, t_6)$

Prove: $\neg alive(Marcus, now)$

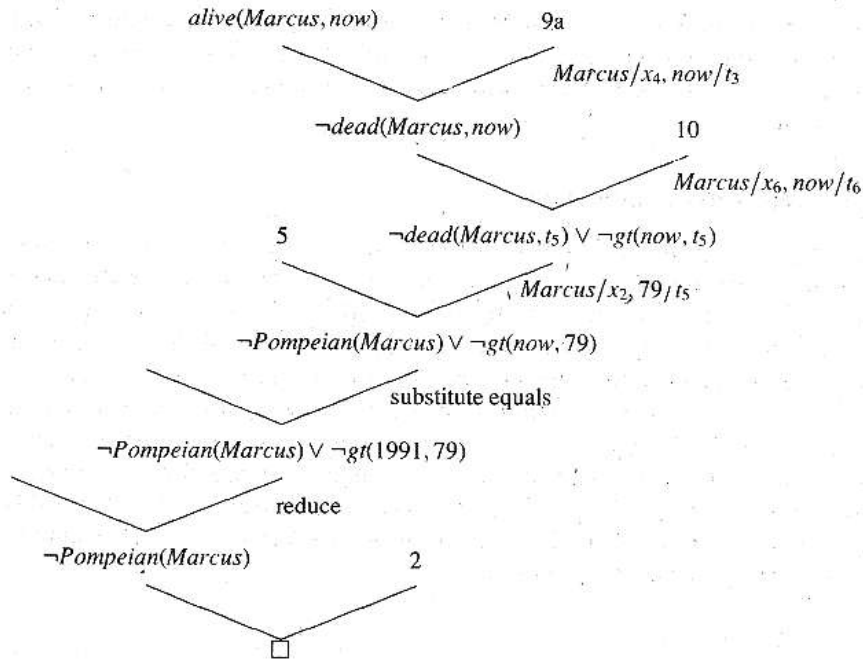


Fig: Using Resolution with Equality and Reduce

4. KNOWLEDGE REPRESENTATION

INTRODUCTION

Knowledge representation is an important issue in both cognitive science & AI. In cognitive science, it is concerned with the way in which information is stored & processed by humans, while in AI, the main focus is on storing knowledge or information in such a manner that programs can process it & achieve human intelligence. In AI, knowledge representation is an important area because intelligent problem solving can be achieved and simplified by choosing an appropriate knowledge representation technique. Representing knowledge in some specific ways makes certain problems easier to solve. The fundamental goal of knowledge representation is to represent knowledge in a manner that facilitates the process of inferencing (drawing conclusions) from it.

Any knowledge representation system should possess properties such as learning, efficiency in acquisition, representational adequacy & inferential adequacy.

- Learning: refers to a capability that acquires new knowledge, behaviours, understanding etc.
- Efficiency in acquisition: refers to the ability to acquire new knowledge using automatic methods wherever possible rather than relying on human intervention.
- Representational adequacy: refers to ability to represent the required knowledge.
- Inferential knowledge: refers to the ability of manipulating knowledge to produce new knowledge from the existing one.

APPROACHES TO KNOWLEDGE REPRESENTATION

Knowledge structures represent objects, facts, relationships & procedures. The main function of these knowledge structures is to provide expertise and information so that a program can operate in an intelligent way. Knowledge structures are usually composed of both traditional & complex structures such as semantic network, frames, scripts, conceptual dependency etc.

1. Relational Knowledge: Relational knowledge comprises objects consisting of attributes & associated values. The simplest way of storing facts is to use a relational method. In this method, each fact is stored in a row of a relational table as done in relational database. A table is defined as a set of data values that is organized using a model of vertical columns identified by attribute names & horizontal rows identified by the corresponding values.

Name	Age (in years)	Sex	Qualification	Salary (in Rupees)
John	38	Male	Graduate	20000
Mike	25	Male	Undergraduate	15000
Mary	30	Female	Ph D	25000
James	29	Male	Graduate	18000